

Queue in Java --- Complete Interview & Exam Notes

Purpose: Complete Queue notes from absolute basics to implementation, dry runs, Java APIs, real-world applications, comparisons, interview answers, and coding practice.

1. What is a Queue?

A **Queue** is a linear data structure / Abstract Data Type (ADT) that follows the **FIFO (First In, First Out)** principle.

FIFO means the element that enters first is normally the element that leaves first.

```
Front                Rear
  ↓                  ↓
[10] [20] [30] [40]
```

10 is the oldest element, so it is removed first. A new element is inserted at the rear.

Real-world analogy

Think about people standing in a ticket-counter line:

```
A → B → C → D
↑
first person served
```

A queue models exactly this waiting-line behavior.

2. Why Do We Need a Queue?

Whenever multiple items are waiting to be processed and **arrival order matters**, a queue is a natural abstraction.

Without a queue, a program would need to manually remember which request arrived first, which one is waiting, and which one should be processed next.

Queues are used for:

- printer jobs
- CPU/task scheduling
- customer-service lines

- network packets
- server requests
- message processing
- producer-consumer systems
- event processing
- Breadth-First Search (BFS)
- buffering

The important idea is:

A queue separates the order in which work arrives from the mechanism that processes the work.

3. Queue Terminology

Front

The **front** identifies the element that will be removed next.

Rear

The **rear** is the insertion end. A new element normally joins the queue here.

Enqueue

Adding an element to the rear.

```
enqueue(50)
```

```
10 → 20 → 30 → 40 → 50
```

Dequeue

Removing the front element.

```
dequeue() → 10
```

```
20 → 30 → 40
```

Peek

Returns the front element without removing it.

```
peek() → 20
```

The queue remains unchanged.

4. Fundamental Queue Operations

Operation Meaning

`enqueue(x)` Add `x` at rear `dequeue()` Remove and return front `peek()` Inspect front without removal `isEmpty()` Check whether queue has no elements `size()` Number of elements `isFull()` Check capacity in a fixed queue

Different programming APIs use different names, but the underlying behavior is the same.

5. FIFO in Detail

Suppose:

```
enqueue(10);
enqueue(20);
enqueue(30);
enqueue(40);
```

Logical queue:

Front								Rear
↓								↓
[10]	[20]	[30]	[40]					

Then:

```
dequeue();
```

returns `10`.

Next dequeue returns `20`, then `30`, then `40`.

Therefore:

```
Insertion order: 10 20 30 40
Removal order:   10 20 30 40
```

This is FIFO.

6. Queue vs Stack

A very common interview question is the difference between Queue and Stack.

Feature Queue Stack

Rule FIFO LIFO Add Rear Top Remove Front Top Example Ticket line Stack of plates
Typical use Scheduling/order processing Undo, recursion, parsing

Queue:

```
10 → 20 → 30
↑
dequeue = 10
```

Stack:

```
10 → 20 → 30
           ↑
         top
pop = 30
```

Interview answer

A queue follows FIFO, meaning the first inserted element is removed first. A stack follows LIFO, meaning the last inserted element is removed first. A queue is suitable when arrival order must be preserved, while a stack is suitable when the most recently added item should be processed first.

7. Queue as an ADT

Queue is an **Abstract Data Type**. An ADT describes the behavior and operations, not one particular physical implementation.

A Queue ADT can be implemented using:

```
Queue ADT
|
+-- Array
+-- Circular Array
+-- Linked List
+-- Deque
+-- Other suitable structures
```

The abstraction says that we need queue behavior such as:

```
enqueue  
  
dequeue  
  
peek  
  
isEmpty  
  
size
```

The implementation decides how those operations are actually performed.

8. Queue Using a Simple Array

Suppose:

```
int[] queue = new int[5];
```

We may initially have:

```
[ ][ ][ ][ ][ ]
```

After insertion:

```
[10][20][30][40][50]
```

A naive deletion from the beginning creates a problem.

If we physically remove `10`, we could shift everything:

```
[20][30][40][50][ ]
```

But four elements moved.

For `n` elements, front deletion can cost **$O(n)$** .

That defeats the purpose of having an efficient queue.

9. The Linear Array Queue Problem

Suppose capacity is `5`:

```
[10][20][30][40][50]
```

Remove three elements:

```
dequeue → 10  
dequeue → 20  
dequeue → 30
```

Logical queue:

```
40 → 50
```

Physical array can look like:

```
[ ][ ][ ][40][50]
```

There is free space at indexes **0** , **1** , and **2** , but a naive linear rear has reached the end.

This is the **wasted-space / false-overflow problem**.

Two obvious solutions are:

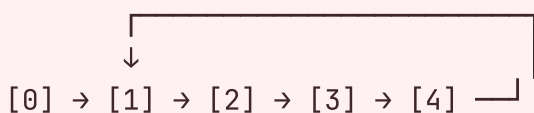
1. shift elements toward the beginning --- but that costs $O(n)$, or
2. reuse the beginning by treating the array as circular.

The second solution gives us a **Circular Queue**.

10. Circular Queue

A circular queue treats the last physical array position as connected to the first.

Conceptually:



After index **4** , the next position is index **0** .

The key formula is:

```
next = (current + 1) % capacity;
```

For capacity **5** :

```
0 → 1 → 2 → 3 → 4 → 0 → 1 → ...
```

This lets the queue reuse positions freed at the beginning without shifting elements.

11. Circular Queue: The Important Formula

For capacity 5 :

```
current = 0 → (0 + 1) % 5 = 1
current = 1 → 2
current = 2 → 3
current = 3 → 4
current = 4 → (4 + 1) % 5 = 0
```

The modulo operator produces the wrap-around.

Next insertion with `front` and `size`

If:

```
front = F
size = S
capacity = C
```

then the next free logical insertion position is:

```
(front + size) % capacity
```

The last existing logical element is at:

```
(front + size - 1) % capacity
```

These formulas are among the most important things to understand for circular queues.

12. Circular Queue Dry Run

Capacity = 5 .

Initial state:

```
data  = [ ][ ][ ][ ][ ]
front = 0
size  = 0
```

enqueue(10)

```
(front + size) % 5  
= (0 + 0) % 5  
= 0
```

```
[10][ ][ ][ ][ ]  
  ↑  
front
```

size = 1.

enqueue(20)

```
(0 + 1) % 5 = 1
```

```
[10][20][ ][ ][ ]
```

size = 2.

enqueue(30)

```
(0 + 2) % 5 = 2
```

```
[10][20][30][ ][ ]
```

size = 3.

dequeue()

Remove 10.

```
[ ][20][30][ ][ ]
```

Move front:

```
front = (front + 1) % 5;
```

so:

```
front = 1  
size = 2
```

dequeue()

Remove 20 .

```
[ ][ ][30][ ][ ]
```

Now:

```
front = 2  
size = 1
```

enqueue(40)

```
(2 + 1) % 5 = 3
```

```
[ ][ ][30][40][ ]
```

enqueue(50)

```
(2 + 2) % 5 = 4
```

```
[ ][ ][30][40][50]
```

enqueue(60)

```
(2 + 3) % 5 = 0
```

Wrap-around occurs:

```
[60][ ][30][40][50]
```

But the logical order is **not** 60 → 30 → 40 → 50 .

Because front = 2 , logical traversal is:

```
index 2 → index 3 → index 4 → index 0
```

```
30 → 40 → 50 → 60
```

This is a critical interview point:

Physical array order and logical queue order can be different in a circular

queue.

13. Complete Fixed-Capacity Circular Queue in Java

```
import java.util.NoSuchElementException;

public class CircularQueue<T> {

    private Object[] data;
    private int front = 0;
    private int size = 0;

    public CircularQueue(int capacity) {
        if (capacity ≤ 0) {
            throw new IllegalArgumentException("Capacity must be positive");
        }
        data = new Object[capacity];
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == data.length;
    }

    public void enqueue(T item) {
        if (isFull()) {
            throw new IllegalStateException("Queue is full");
        }

        int rear = (front + size) % data.length;
        data[rear] = item;
        size++;
    }

    @SuppressWarnings("unchecked")
    public T dequeue() {
        if (isEmpty()) {
            throw new NoSuchElementException("Queue is empty");
        }
    }
}
```

```

        T item = (T) data[front];
        data[front] = null;

        front = (front + 1) % data.length;
        size--;

        return item;
    }

    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) {
            throw new NoSuchElementException("Queue is empty");
        }
        return (T) data[front];
    }
}

```

14. Circular Queue Code --- Line-by-Line Logic

Constructor

```
data = new Object[capacity];
```

Creates the underlying array.

Initial state:

```
front = 0
size = 0
```

size()

```
return size;
```

Because size is maintained directly, this is $O(1)$.

isEmpty()

```
return size == 0;
```

If no logical elements exist, the queue is empty.

`isFull()`

```
return size == data.length;
```

In a fixed-capacity queue, this means there is no free position.

15. `enqueue()` --- Detailed Logic

```
public void enqueue(T item) {  
    if (isFull()) {  
        throw new IllegalStateException("Queue is full");  
    }  
  
    int rear = (front + size) % data.length;  
    data[rear] = item;  
    size++;  
}
```

Step 1 --- Check overflow

```
if (isFull())
```

Prevents insertion when the fixed queue is full.

Step 2 --- Calculate insertion position

```
int rear = (front + size) % data.length;
```

The modulo makes the position wrap around.

Step 3 --- Store

```
data[rear] = item;
```

Step 4 --- Update logical size

```
size++;
```

Only one element was added, so size increases by one.

16. `dequeue()` --- Detailed Logic

```
T item = (T) data[front];
data[front] = null;
front = (front + 1) % data.length;
size--;
return item;
```

The order is important:

1. read the front element,
2. clear its array slot,
3. move front circularly,
4. decrease size,
5. return the removed element.

Clearing the slot is useful when the queue stores object references because it prevents an already-removed object from being unnecessarily retained by the backing array.

17. `peek()` VS `dequeue()`

`peek()` :

```
Queue: 10 20 30
peek() → 10
Queue: 10 20 30
```

`dequeue()` :

```
Queue: 10 20 30
dequeue() → 10
Queue: 20 30
```

Remember:

Peek observes; dequeue modifies.

18. Linked-List Queue

A queue can also be implemented using linked nodes.

Maintain two references:

```
front                rear
  ↓                  ↓
[10] → [20] → [30] → [40] → null
```

Enqueue

Insert at rear :

```
[10] → [20] → [30] → [40] → [50]
                                ↑
                              rear
```

Dequeue

Remove from front :

```
front
  ↓
[20] → [30] → [40] → [50]
```

With both references, enqueue and dequeue can both be $O(1)$.

19. Why a Linked Queue Needs Both Front and Rear

If only front is maintained, finding the last node requires traversal:

```
front → node → node → node → last
```

That makes enqueue $O(n)$.

With rear :

```
rear.next = newNode;
rear = newNode;
```

No traversal is required.

Therefore:

```
enqueue =  $O(1)$ 
dequeue =  $O(1)$ 
```


20. Complete Linked Queue in Java

```
import java.util.NoSuchElementException;

public class LinkedQueue<T> {

    private static class Node<T> {
        T data;
        Node<T> next;

        Node(T data) {
            this.data = data;
        }
    }

    private Node<T> front;
    private Node<T> rear;
    private int size;

    public boolean isEmpty() {
        return size == 0;
    }

    public int size() {
        return size;
    }

    public void enqueue(T value) {
        Node<T> newNode = new Node<>(value);

        if (isEmpty()) {
            front = rear = newNode;
        } else {
            rear.next = newNode;
            rear = newNode;
        }

        size++;
    }

    public T dequeue() {
        if (isEmpty()) {
            throw new NoSuchElementException("Queue is empty");
        }
    }
}
```

```

        T value = front.data;
        front = front.next;
        size--;

        if (size == 0) {
            rear = null;
        }

        return value;
    }

    public T peek() {
        if (isEmpty()) {
            throw new NoSuchElementException("Queue is empty");
        }
        return front.data;
    }
}

```

Important edge case

If there is one node:

```
front → [10] ← rear
```

After dequeue, both must become `null`:

```
front = null
rear = null
```

Otherwise the queue's internal state becomes inconsistent.

21. Array Queue vs Linked Queue

Feature Circular Array Linked Queue

Capacity	Fixed	unless resized	Dynamic	Storage	Contiguous	Separate nodes	Enqueue	O(1)	O(1)	Dequeue	O(1)	O(1)	Resize	May be needed	Not needed	Node overhead	None	Yes
Cache locality	Usually better	Usually worse	Predictable	memory	Good	Less predictable												

There is no universally best implementation.

Prefer circular array when

- capacity is known or bounded,
- contiguous storage is useful,
- cache locality matters,
- predictable memory usage matters.

Prefer linked queue when

- size is highly dynamic,
 - fixed capacity is inconvenient,
 - dynamic node-based growth is acceptable.
-

22. Java `Queue` Interface

Java provides:

```
java.util.Queue<E>
```

It is an interface representing queue-like behavior.

Common implementations include:

```
ArrayDeque  
LinkedList  
PriorityQueue
```

But they do not all have identical ordering semantics. In particular, `PriorityQueue` is priority-based rather than ordinary FIFO.

23. Java Queue Method Pairs

Purpose	Exception-oriented method	Special-value method
---------	---------------------------	----------------------

Insert	<code>add(e)</code>	<code>offer(e)</code>	Remove	<code>remove()</code>	<code>poll()</code>	Examine	<code>element()</code>	<code>peek()</code>
--------	---------------------	-----------------------	--------	-----------------------	---------------------	---------	------------------------	---------------------

`add()` **VS** `offer()`

Both attempt insertion. `offer()` communicates inability to insert using a return value in bounded situations, while `add()` uses exception-oriented behavior.

remove() VS poll()

If empty:

```
remove() → NoSuchElementException  
poll()   → null
```

element() VS peek()

If empty:

```
element() → NoSuchElementException  
peek()    → null
```

This is a very common Java viva question.

24. **ArrayDeque as a Queue**

```
import java.util.ArrayDeque;  
import java.util.Queue;  
  
public class Demo {  
    public static void main(String[] args) {  
        Queue<Integer> q = new ArrayDeque<>();  
  
        q.offer(10);  
        q.offer(20);  
        q.offer(30);  
  
        System.out.println(q.peek()); // 10  
        System.out.println(q.poll()); // 10  
        System.out.println(q.poll()); // 20  
        System.out.println(q.peek()); // 30  
    }  
}
```

Logical behavior:

```
offer(10)
offer(20)
offer(30)
```

10 → 20 → 30

```
poll() → 10
```

20 → 30

25. Queue and Deque

Deque means **double-ended queue**.

It supports operations at both ends:

```
addFirst
addLast
removeFirst
removeLast
peekFirst
peekLast
```

Conceptually:

front	rear
↓	↓
[10] [20] [30] [40]	
↑	↑
operations	operations

A deque is more general than a normal queue.

If we restrict it to:

```
insert at rear
remove at front
```

it behaves as a Queue.

If we restrict it to:

```
insert at one end
remove from same end
```

it behaves as a Stack.

26. Your CircularDeque Lab and Queue

Your uploaded `CircularDeque.java` uses:

```
private Object[] data;  
private int front = 0;  
private int size = 0;
```

and explicitly uses circular-array formulas for both ends. Its `addLast` uses:

```
data[(front + size) % data.length] = item;
```

and its `removeFirst` reads `data[front]`, clears that slot, advances `front` using modulo arithmetic, and decrements `size`. `fileciteÖturn0file3ÖL5-L10Ö`
`fileciteÖturn0file3ÖL29-L43Ö`

This is directly applicable to understanding an array-based Queue.

Its `grow()` method copies the logical elements into a larger array in logical order and resets `front` to zero. The copy is $O(n)$, while normal end operations remain $O(1)$; occasional growth leads to amortized $O(1)$ insertion. `fileciteÖturn0file3ÖL59-L64Ö`

The same structure is demonstrated as both a stack and a queue in the uploaded main method. `fileciteÖturn0file3ÖL75-L90Ö`

27. Circular Growth and Amortized $O(1)$

Suppose a dynamic circular array is full.

Growing from capacity `n` to a larger array requires copying the existing elements:

```
0(n)
```

That particular operation is expensive.

But growth happens only occasionally. Most insertions simply write into the next available position:

```
0(1)
```

Over a long sequence of operations, the average cost per insertion is:

Amortized $O(1)$

Important interview wording:

Amortized $O(1)$ does not mean every individual operation is $O(1)$. It means the total cost spread across a sequence of operations gives constant average cost.

28. Overflow and Underflow

Overflow

In a fixed-capacity queue:

```
capacity = 5  
size = 5
```

Trying another insertion causes overflow.

Underflow

Trying to remove from:

```
size = 0
```

causes underflow.

The exact behavior depends on the API: an implementation may throw an exception or return a special value such as `null`.

29. Queue Complexity

For a good implementation:

Operation Circular Array Linked Queue

Enqueue $O(1)$ $O(1)$ Dequeue $O(1)$ $O(1)$ Peek $O(1)$ $O(1)$ `isEmpty` $O(1)$ $O(1)$ `size` $O(1)$ if maintained $O(1)$ if maintained Dynamic resize $O(n)$ occasionally Not needed

Space:

```
 $O(n)$ 
```

for n stored elements, with implementation-specific capacity/overhead.

30. Why Shifting Should Be Avoided

Suppose:

```
[10][20][30][40][50]
```

Deleting 10 by shifting gives:

```
[20][30][40][50][ ]
```

Four elements moved.

If there are n elements, this can be $O(n)$.

A circular queue instead performs:

```
front = (front + 1) % capacity;
```

Only an index changes.

Therefore:

```
physical shifting →  $O(n)$   
logical index movement →  $O(1)$ 
```

This is the main reason circular queues are so useful.

31. Queue Invariants

An **invariant** is a condition that should remain true in every valid state.

For a circular queue with capacity C :

```
 $0 \leq \text{size} \leq C$ 
```

If:

```
size == 0
```

queue is empty.

If:

`size = C`

fixed queue is full.

The logical element at offset `k` is located at:

`(front + k) % C`

for:

`0 ≤ k < size`

Thinking in invariants is very useful when debugging a queue implementation.

32. Queue Applications

Printer queue

Document A
Document B
Document C

Printer can process:

`A → B → C`

Customer service

`Customer A → B → C → D`

Normally A is served before B, etc.

Network buffering

Packets may arrive faster than they can be processed:

Packet A → Packet B → Packet C
↓
Queue/Buffer

Server requests

Incoming requests can wait for available workers.

Producer-consumer

```
graph TD
    P[Producer] --> Q[Queue / Buffer]
    Q --> C[Consumer]
```

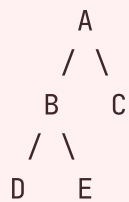
The producer adds work; the consumer removes and processes it.

BFS

BFS uses a queue to process vertices level by level.

33. BFS Example

Graph:



Start at A.

Queue: [A]

Remove A and add B,C:

Queue: [B,C]

Remove B and add D,E:

Queue: [C,D,E]

Remove C:

Queue: [D,E]

Traversal is:

A → B → C → D → E

The queue guarantees that earlier-discovered nodes are processed before later pending nodes.

34. BFS Code

```
import java.util.*;

public class BFS {

    static void bfs(Map<Integer, List<Integer>> graph, int start) {

        Queue<Integer> q = new ArrayDeque<>();
        Set<Integer> visited = new HashSet<>();

        q.offer(start);
        visited.add(start);

        while (!q.isEmpty()) {

            int current = q.poll();
            System.out.println(current);

            for (int neighbor : graph.getOrDefault(current, List.of())) {

                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    q.offer(neighbor);
                }
            }
        }
    }
}
```

Logic

`offer(start)` puts the first vertex into the waiting line.

`poll()` removes the oldest pending vertex.

Unvisited neighbors are added to the rear.

That repeated FIFO behavior creates level-by-level traversal.

35. PriorityQueue Is Not a Normal FIFO Queue

This is a major interview trap.

Normal FIFO queue:

```
enqueue(30)
enqueue(10)
enqueue(20)

dequeue → 30
```

A priority queue may return `10` first under natural ordering:

```
add(30)
add(10)
add(20)

poll() → 10
```

Therefore:

```
Queue      → arrival order
PriorityQueue → priority/order
```

Do not say that every Java class implementing `Queue` must be ordinary FIFO. The interface permits different ordering semantics depending on implementation.

36. Queue vs ArrayList

`ArrayList` is designed for indexed collection behavior.

```
list.get(i)
```

is typically $O(1)$.

But removing the first element requires shifting the remaining elements:

```
remove(0) →  $O(n)$ 
```

A Queue is specifically designed around insertion at one end and removal at the other, and good implementations provide $O(1)$ end operations.

So:

Do not use `ArrayList.remove(0)` as your default queue implementation when frequent front removal is required.

37. Queue vs LinkedList in Java

Java `LinkedList` implements `Deque`, so it can also be used through the `Queue` interface:

```
Queue<Integer> q = new LinkedList<>();
```

However, for many ordinary queue/deque use cases, `ArrayDeque` is generally a strong default because contiguous array storage avoids the per-element node overhead of a linked structure and generally provides better locality.

The correct interview answer is not that linked lists are "bad". The correct answer is to compare the workload and trade-offs.

38. Which Implementation Is Better?

There is no universal winner.

Circular array is attractive when

- maximum capacity is known/bounded,
- contiguous storage is useful,
- memory locality matters,
- predictable storage is important.

Linked queue is attractive when

- the size is highly dynamic,
- fixed capacity is inconvenient,
- node-based dynamic growth is acceptable.

`ArrayDeque` is often a strong Java choice when

- you need a normal in-memory queue/deque,
- you want efficient operations at both ends,
- dynamic resizing is useful,

- you do not need special concurrent queue semantics.

A strong interview response is:

"The best implementation depends on capacity requirements, memory locality, dynamic growth, and concurrency requirements."

39. Queue Using Two Stacks

Classic interview problem.

Use:

```
input stack
output stack
```

Enqueue

Push into `input` .

Dequeue

If `output` is empty, move all elements from `input` to `output` .

Example:

```
enqueue 10
enqueue 20
enqueue 30
```

Input stack conceptually contains:

```
30
20
10
```

Transfer to output:

```
10
20
30
```

Now popping output gives:

10 → 20 → 30

which is FIFO.

40. Queue Using Two Stacks --- Code

```
import java.util.ArrayDeque;
import java.util.Deque;

public class QueueUsingStacks<T> {

    private Deque<T> input = new ArrayDeque<>();
    private Deque<T> output = new ArrayDeque<>();

    public void enqueue(T value) {
        input.push(value);
    }

    private void moveIfNeeded() {
        if (output.isEmpty()) {
            while (!input.isEmpty()) {
                output.push(input.pop());
            }
        }
    }

    public T dequeue() {
        moveIfNeeded();

        if (output.isEmpty()) {
            throw new IllegalStateException("Queue is empty");
        }

        return output.pop();
    }

    public T peek() {
        moveIfNeeded();

        if (output.isEmpty()) {
            throw new IllegalStateException("Queue is empty");
        }

        return output.peek();
    }

    public boolean isEmpty() {
        return input.isEmpty() && output.isEmpty();
    }
}
```

```
}  
}
```

Complexity

A single transfer can cost $O(n)$, but each element is moved only when necessary. Over a long sequence, enqueue/dequeue are **amortized $O(1)$** .

41. Producer-Consumer and Queue as a Buffer

A queue can decouple two components:

```
Fast Producer  
  ↓  
+-----+  
| Queue/Buffer |  
+-----+  
  ↓  
Slow Consumer
```

If the producer temporarily becomes faster, work can wait in the queue instead of being immediately lost or forcing the producer to perform all processing itself.

This is why queues are fundamental in asynchronous systems, messaging, networking, and task processing.

Real systems may use bounded queues, multiple consumers, priorities, backpressure, and concurrency controls. A simple FIFO queue is the foundational abstraction.

42. Important Edge Cases

Empty queue

```
size = 0
```

Check before dequeue/peek according to the API.

One element

```
front → [10] ← rear
```

After dequeue:

```
front = null
rear = null
```

for a linked implementation.

Full fixed queue

```
[10][20][30][40][50]
```

Another insertion must not overwrite valid data.

Wrap-around

Test cases where the insertion index reaches the last physical slot and then returns to index zero.

Repeated empty/non-empty transitions

Repeatedly enqueueing and dequeueing is a good way to expose incorrect state management.

43. Common Mistakes

1. Confusing FIFO with LIFO.
 2. Removing from index zero of an array and forgetting shifting cost.
 3. Forgetting circular wrap-around.
 4. Forgetting the full condition in a fixed queue.
 5. Forgetting the empty condition.
 6. Losing track of `size`.
 7. Incorrectly updating `front` or `rear` after the last linked-node removal.
 8. Assuming physical circular-array order equals logical queue order.
 9. Calling `PriorityQueue` a normal FIFO queue.
 10. Saying one implementation is always best without considering requirements.
-

44. Queue Operation Complexity

Operation Circular Array Linked Queue

Enqueue $O(1)$ $O(1)$ Dequeue $O(1)$ $O(1)$ Peek $O(1)$ $O(1)$ `isEmpty` $O(1)$ $O(1)$ `size` $O(1)$ if stored $O(1)$ if stored Resize $O(n)$ occasionally Not required

Space is $O(n)$ for n stored elements, with implementation-specific capacity and overhead.

45. Interview Questions and Strong Answers

Q1. What is a Queue?

A queue is a linear data structure and abstract data type that follows FIFO, or First In First Out. Elements are normally inserted at the rear and removed from the front. Its fundamental operations are enqueue, dequeue, peek, isEmpty, and size. It can be implemented using arrays, circular arrays, linked lists, and other structures.

Q2. Why is a circular queue needed?

A simple fixed-size linear array queue can waste space after front elements are removed. The rear can reach the last index even though there are free positions at the beginning. A circular queue treats the array as a ring and reuses those positions through modulo arithmetic, avoiding unnecessary shifting and allowing $O(1)$ enqueue and dequeue.

Q3. Why not shift after every dequeue?

Shifting all remaining elements after each deletion can require $O(n)$ work. A circular queue instead changes the logical front index, so deletion only requires constant-time index updates.

Q4. How do you calculate the next insertion position?

With `front` and `size`, the next insertion position is $(\text{front} + \text{size}) \% \text{capacity}$. The modulo handles wrap-around.

Q5. Why maintain `size`?

It gives the number of logical elements directly and makes empty/full checks straightforward. In a circular array, physical front and rear positions can

wrap, so an additional state rule or size variable is useful to distinguish empty and full states.

Q6. What is overflow?

Overflow is an attempt to insert into a full fixed-capacity queue. Dynamic implementations may resize instead.

Q7. What is underflow?

Underflow is an attempt to remove from an empty queue.

Q8. Queue vs Deque?

A queue normally inserts at one end and removes at the other, producing FIFO behavior. A deque supports insertion and removal at both ends. A deque can therefore be restricted to behave like a queue or a stack.

Q9. Queue vs PriorityQueue?

A normal FIFO queue removes by arrival order. A priority queue removes according to priority/order rules, so the earliest inserted item is not necessarily removed first.

Q10. Array queue vs linked queue?

A circular array offers contiguous storage and usually better cache locality and memory efficiency when capacity is known. A linked queue grows dynamically but has node/reference overhead and generally poorer locality. Both can provide $O(1)$ enqueue and dequeue when implemented correctly.

Q11. Why is `ArrayDeque` useful?

`ArrayDeque` is a resizable-array deque implementation that supports efficient operations at both ends. It can therefore act as a queue or stack and is a strong general-purpose choice for many non-concurrent use cases.

Q12. Can a queue be implemented using two stacks?

Yes. Enqueue into one stack. For dequeue, if the second stack is empty, transfer all elements from the first stack to the second. This reverses the order, making the oldest element accessible first. The operations are amortized $O(1)$.

46. Rapid Viva Questions

What principle does Queue follow?

FIFO.

Where is insertion performed?

Rear.

Where is deletion performed?

Front.

What is enqueue?

Insertion into the queue.

What is dequeue?

Removal from the front.

What is peek?

Inspect front without removing.

What is underflow?

Removal from an empty queue.

What is overflow?

Insertion into a full fixed-capacity queue.

Why circular queue?

To reuse free positions without shifting elements.

Circular next-index formula?

```
(index + 1) % capacity
```

Circular next-insertion formula?

```
(front + size) % capacity
```

Typical enqueue complexity?

$O(1)$.

Typical dequeue complexity?

$O(1)$.

Queue vs Stack?

FIFO vs LIFO.

What is a Deque?

Double-ended queue.

Is PriorityQueue FIFO?

No.

47. Coding Practice

Problem 1 --- Circular Queue

Implement a fixed-capacity circular queue with:

```
enqueue()  
dequeue()  
peek()  
isEmpty()  
isFull()  
size()
```

Requirements:

- no shifting,
- use modulo arithmetic,

- detect overflow,
- detect underflow.

Problem 2 --- Dry Run

Given:

```
capacity = 5

enqueue(10)
enqueue(20)
enqueue(30)
dequeue()
dequeue()
enqueue(40)
enqueue(50)
enqueue(60)
```

Find:

1. final physical array,
2. final `front` ,
3. final `size` ,
4. logical queue order.

Problem 3 --- Linked Queue

Implement a queue with:

```
front
rear
size
```

Make sure the one-element-to-empty transition is correct.

Problem 4 --- Two Stacks

Implement Queue using two stacks and explain why the removal order becomes FIFO.

Problem 5 --- BFS

Given a graph adjacency list, perform BFS using a Queue and explain why FIFO is necessary for level-order traversal.

48. 2--3 Minute Interview Explanation

If an interviewer says **"Explain Queue"**, a strong response is:

A queue is a linear data structure and abstract data type that follows the FIFO, or First In First Out, principle. The main idea is that elements are inserted at the rear and removed from the front, so the element that arrives first is normally processed first. The main operations are enqueue for insertion, dequeue for removal, peek for examining the front element, along with isEmpty and size.

A queue can be implemented using an array, circular array, linked list, or other structures. A simple linear array implementation has a problem: after removing elements from the front, empty positions can remain unused while the rear reaches the end. Shifting all elements after every deletion would make dequeue $O(n)$. A circular queue solves this by treating the array as a ring and using modulo arithmetic. For example, $(\text{index} + 1) \% \text{capacity}$ performs wrap-around, and with front and size, $(\text{front} + \text{size}) \% \text{capacity}$ gives the next insertion position. This allows $O(1)$ enqueue and dequeue in a fixed-capacity circular implementation.

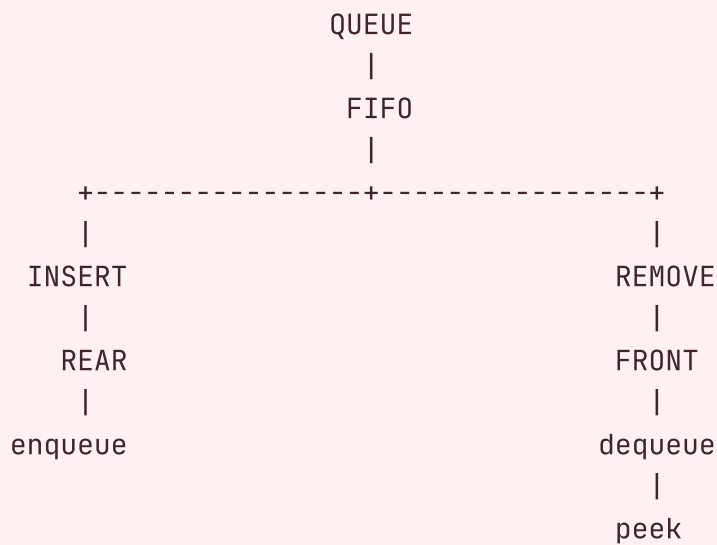
Another implementation is a linked queue, where we maintain both front and rear references. Enqueue adds a node at rear and dequeue removes a node from front, giving $O(1)$ operations without fixed array capacity, although linked nodes have additional memory overhead and generally poorer cache locality.

In Java, the Queue interface provides queue operations, and ArrayDeque is commonly used for efficient queue/dequeue behavior. Java also provides method pairs such as add versus offer, remove versus poll, and element versus peek, which differ mainly in how failure or empty conditions are reported.

Queues are used in scheduling, buffering, request processing, producer-consumer systems, printer queues, networking, and graph algorithms such as BFS. A deque is more general because it supports both ends, while a priority queue removes according to priority rather than ordinary FIFO order.

So the key idea is: Queue provides FIFO ordering; circular queues efficiently reuse array space; linked queues provide dynamic storage; and the best implementation depends on capacity, memory, performance, and application requirements.

49. Final Mental Model



Circular array:

```
next position:
(index + 1) % capacity

next insertion:
(front + size) % capacity

logical element k:
(front + k) % capacity
```

Linked queue:

```
front → first node → ... → last node ← rear
```

Java:

```
Queue<Integer> q = new ArrayDeque<>();

q.offer(x);
q.poll();
q.peek();
q.isEmpty();
q.size();
```

50. One-Page Revision

QUEUE

=====

Definition:

Linear ADT following FIFO.

FIFO:

First In → First Out

Main operations:

enqueue → rear

dequeue → front

peek → front without removal

Errors:

overflow → insert into full fixed queue

underflow → remove from empty queue

Implementations:

1. Array
2. Circular Array
3. Linked List
4. Deque-based implementation

Linear-array problem:

Front deletions can leave wasted space.

Circular queue:

Reuse beginning positions.

Formula:

$\text{next} = (\text{index} + 1) \% \text{capacity}$

With front + size:

$\text{insertion} = (\text{front} + \text{size}) \% \text{capacity}$

Complexity:

enqueue → $O(1)$

dequeue → $O(1)$

peek → $O(1)$

size → $O(1)$ if tracked

resize → $O(n)$ occasionally, amortized $O(1)$

Linked queue:

front + rear

enqueue $\rightarrow O(1)$

dequeue $\rightarrow O(1)$

Java:

Queue<E>

ArrayDeque<E>

Method pairs:

add / offer

remove / poll

element / peek

Queue:

FIFO

Stack:

LIFO

Deque:

both ends

PriorityQueue:

priority-based ordering

Applications:

BFS

Scheduling

Printer queue

Networking

Buffering

Producer-consumer

Request processing

51. Final Takeaway

The best way to understand Queue is to understand the design problem rather than memorizing definitions:

Need ordered waiting



Queue



FIFO



insert at rear

remove at front



array implementation



front deletion can cause shifting/wasted space



circular array



wrap-around using modulo



$O(1)$ enqueue/dequeue

Then understand the implementation trade-off:

Circular Array



fast + compact + cache-friendly

but capacity/resizing must be managed

Linked Queue



dynamic size

but node/reference overhead

ArrayDeque



strong general-purpose Java choice

for many queue/deque use cases

PriorityQueue



not ordinary FIFO

uses priority/order